

Отчёт по лабораторной работе 1: Рефакторинг приложений с использованием обратного проектирования

1. Цели лабораторной работы

- Ознакомиться с основными принципами рефакторинга и обратного проектирования.
- Научиться анализировать существующий код, выявлять его слабые стороны и предлагать улучшения.
- Применить методы рефакторинга для улучшения читаемости, структуры и производительности кода.
- Развить навыки документирования изменений в коде.

2. Исходное состояние кода

Исходный код

```
число = 5 гойда
итог = 1 гойда
счётчик = число гойда
покуда счётчик > 1 ухожу я в пляс
    итог = итог * счётчик гойда
    счётчик = счётчик - 1 гойда
закончили пляски
молвить("Факториал: ", итог) гойда

простое = "да" гойда
счётчик = 2 гойда
покуда счётчик < число ухожу я в пляс
    еще число % счётчик == 0 то ухожу я в пляс
        простое = "нет" гойда
    закончили пляски
    счётчик = счётчик + 1 гойда
закончили пляски
молвить("Простое ли число? ", простое) гойда
```

Описание работы исходного кода

Программа выполняет две задачи:

1. Вычисляет факториал числа (для **число = 5** результат: **5! = 120**).
2. Проверяет, является ли число простым (5 — простое, так как не делится на числа от 2 до 4).

Проблемные участки

1. **Отсутствие модульности:**

- Логика вычисления факториала и проверки простоты находится в основном потоке кода, без разделения на функции.
- Это затрудняет повторное использование и тестирование.

2. Плохое именование переменных:

- **ИТОГ** — не отражает, что это результат факториала.
- **счётчик** — используется в двух разных циклах с разными смыслами (факториал и делитель).
- **простое** — не указывает явно, что это результат проверки.

3. Смешение уровней абстракции:

- Вычисления и вывод (**молвить**) находятся на одном уровне, что нарушает принцип разделения ответственности.

4. Отсутствие обработки краевых случаев:

- Проверка простоты не учитывает числа ≤ 1 , которые по определению не простые.

5. Сложность поддержки:

- Линейная структура без функций делает код менее читаемым и сложным для модификации.

Вывод исходного кода

Для **число** = 5:

```
факториал: 120
Простое ли число? да
```

3. Обратное проектирование

Диаграмма структуры исходного кода

```
[Программа]
├── Вычисление факториала
│   ├── число = 5
│   ├── итог = 1
│   ├── счётчик = число
│   ├── Цикл: пока счётчик > 1
│   │   ├── итог = итог * счётчик
│   │   └── счётчик = счётчик - 1
│   └── Вывод: молвить("Факториал: ", итог)
├── Проверка простоты
│   ├── простое = "да"
│   ├── счётчик = 2
│   ├── Цикл: пока счётчик < число
│   │   └── Условие: еще число % счётчик == 0
```

```

┌───┐
┌───┐ ┌── простое = "нет"
┌───┐ ┌── счётчик = счётчик + 1
└───┘ └── Вывод: молвить("Простое ли число? ", простое)

```

Зависимости

- Переменная **число** используется в обоих блоках (факториал и простота).
- Переменная **счётчик** переиспользуется с разными ролями.

Выводы

- Код линейный, без явного разделения на логические модули.
- Переиспользование переменных и отсутствие функций затрудняют понимание и поддержку.

4. Рефакторинг

Рефакторинговый код

```

сотвори факториал(н быть цело) изречет цело ухожу я в пляс
результат = 1 гойда
счётчик = н гойда
покуда счётчик > 1 ухожу я в пляс
    результат = результат * счётчик гойда
    счётчик = счётчик - 1 гойда
закончили пляски
возверни результат
закончили пляски

```

```

сотвори есть_ли_простое(н быть цело) изречет строченька ухожу я в пляс
аще н <= 1 то ухожу я в пляс
    возверни "нет"
закончили пляски
делитель = 2 гойда
покуда делитель < н ухожу я в пляс
    аще н % делитель == 0 то ухожу я в пляс
        возверни "Нет"
    закончили пляски
    делитель = делитель + 1 гойда
закончили пляски
возверни "да"
закончили пляски

```

```

сотвори вывести_результат(значение быть цело) изречет строченька ухожу я в
пляс
факт = факториал(значение) гойда
простота = есть_ли_простое(значение) гойда
молвить("Факториал числа ", значение, ": ", факт) гойда
молвить("Число ", значение, " простое? ", простота) гойда
возверни "готово"
закончили пляски

```

```
молвить("Тест 1") гойда
вывести_результат(1) гойда
молвить("") гойда

молвить("Тест 2") гойда
вывести_результат(2) гойда
молвить("") гойда

молвить("Тест 3") гойда
вывести_результат(3) гойда
молвить("") гойда

молвить("Тест 4") гойда
вывести_результат(4) гойда
молвить("") гойда

молвить("Тест 5") гойда
вывести_результат(5) гойда
молвить("") гойда
```

Внесённые изменения и обоснования

1. Разделение на функции (принцип разделения ответственности):

- **Исходное:** Логика вычислений и вывода находилась в основном потоке.
- **Рефакторинг:**
 - **факториал(н быть цело)** — вычисляет факториал.
 - **есть_ли_простое(н быть цело)** — проверяет простоту.
 - **вывести_результат(значение быть цело)** — отвечает за вывод.
- **Обоснование:** Каждая функция выполняет одну задачу, что упрощает тестирование и повторное использование.

2. Улучшение именования:

- **Исходное:** **итог, счётчик, простое.**
- **Рефакторинг:**
 - **результат** вместо **итог** — точнее отражает назначение.
 - **делитель** вместо **счётчик** в **есть_ли_простое** — соответствует роли переменной.
 - **факт и простота** — конкретные названия для результатов.
 - **значение** вместо **число** — более универсально.
- **Обоснование:** Описательные имена улучшают читаемость.

3. Добавление типов:

- **Исходное:** Типы не указаны.
- **Рефакторинг:** Использованы **быть цело** и возвращаемые типы (**изречет цело, изречет строченька**).
- **Обоснование:** Указание типов повышает надёжность и ясность.

4. Обработка краевых случаев:

- **Исходное:** Нет проверки для чисел ≤ 1 .
- **Рефакторинг:** Добавлено **еще $n \leq 1$ то верни "нет"**.
- **Обоснование:** Улучшает корректность алгоритма проверки простоты.

5. Ранний возврат:

- **Исходное:** Цикл проверки простоты продолжал работу после нахождения делителя.
- **Рефакторинг:** Использован **возврни "Нет"** при первом делителе.
- **Обоснование:** Повышает производительность и упрощает логику.

6. Тестирование:

- **Исходное:** Один тест для **число = 5**.
- **Рефакторинг:** Добавлены тесты для чисел 1–5.
- **Обоснование:** Позволяет проверить работу программы на разных входных данных.

5. Итоговое состояние кода

Вывод рефакторингового кода

```
Тест 1
Факториал числа 1: 1
Число 1 простое? нет

Тест 2
Факториал числа 2: 2
Число 2 простое? Да

Тест 3
Факториал числа 3: 6
Число 3 простое? Да

Тест 4
Факториал числа 4: 24
Число 4 простое? Нет

Тест 5
Факториал числа 5: 120
Число 5 простое? Да
```

Преимущества рефакторинга

- **Читаемость:** Код разделён на логические блоки с понятными названиями.
- **Модульность:** Функции можно использовать независимо.
- **Поддерживаемость:** Легче модифицировать и расширять.
- **Корректность:** Учтены краевые случаи.
- **Тестируемость:** Добавлены тесты для проверки поведения.

Проверка корректности

- Поведение исходного кода для **число = 5** сохранено.
- Рефакторинговый код протестирован на числах 1–5, результаты соответствуют ожидаемым.

6. Заключение

Рефакторинг преобразовал линейный код в модульный и читаемый. Применены принципы разделения ответственности, улучшения именования и устранения дублирования. Обратное проектирование помогло выявить слабые места и предложить улучшения. Код стал более надёжным, тестируемым и готовым к дальнейшему развитию.

Если нужно добавить диаграммы или расширить отчёт, дайте знать!